

有数据包

定位X-Sign的实现

```

})();
exports.request = c, exports.upload = function (e, r) {
  var t = o.useAuth(),
    u = s.useClient(),
    c = a.sign("dir=" + e);
  return Date.now().toString().slice(0, 11), new Promise((function (a, o) {
    n.index.uploadFile({
      url: "".concat(i.config.base_url, "/miniprogram/common/material/upload"),
      name: "file",
      filePath: r,
      header: {
        "X-Sign": c.sign,
        "X-Timestamp": c.timestamp,
        "X-Version-Code": u.getClientByKey("version_code") || "",
        "X-Version-Name": u.getClientByKey("version_name") || "",
        "X-Device-Id": u.getClientByKey("device_id") || "",
        Accept: "application/json",
        Authorization: "Bearer " + t.token
      },
      formData: {
define("common/interceptor.js", function (require, module, exports, window, document, frames, self, location,
"use strict";
var e = require("../@babel/runtime/helpers/regeneratorRuntime"),
  r = require("../@babel/runtime/helpers/asyncToGenerator"),
  t = require("./vendor.js"),
  n = require("./config.js"),
  a = require("../stores/useAuth.js"),
  i = require("../stores/useClient.js"),
  s = require("../sign.js"),
  u = function () {
    var u = r(e).mark((function u(o) {
      var c, p;
      return e().wrap((function (u) {
        for (;;) switch (u.prev = u.next) {
          case 0:
            c = a.useAuth(), p = i.useClient(), t.index.addInterceptor("request", {
              invoke: function (e) {
                var r = {
                  Accept: "application/json"
                };
                r.Authorization = "Bearer " + c.token, e.data || (e.data = {});
                var t = JSON.stringify(e.data),
                  a = s.sign(t);
                r["X-Timestamp"] = a.timestamp, r["X-Sign"] = a.sign, r["X-Version-Code"]

```

查看sign.js中定义的算法实现

```
define("common/sign.js", function (require, module, exports, window, document, frames, self, location, navigator) {
    "use strict";
    var t = require("../vendor.js");
    exports.sign = function (n) {
        var r = Date.now().toString().slice(0, 11);
        return {
            sign: t.Md5(t.sha256("".concat(n, "&").concat(r))).toString(t.CryptoJS.enc.Hex),
            timestamp: r
        };
    };
}, {
    isPage: false,
    isComponent: false,
    currentFile: 'common/sign.js'
});
```

interceptor拦截器会被定义在request中

```
define("common/interceptor.js", function (require, module, exports, window, document, frames, self, location, navigator) {
    "use strict";
    var e = require("../@babel/runtime/helpers/regeneratorRuntime"),
        r = require("../@babel/runtime/helpers/asyncToGenerator"),
        t = require("../vendor.js"),
        n = require("../config.js"),
        a = require("../stores/useAuth.js"),
        i = require("../stores/useClient.js"),
        s = require("../sign.js"),
        u = function () {
            var u = r(e()).mark(function u(o) {
                var c, p;
                return e().wrap(function (u) {
                    for (;;) switch (u.prev = u.next) {
                        case 0:
                            c = a.useAuth(), p = i.useClient(), t.index.addInterceptor("request", {
                                invoke: function (e) {
                                    var r = {
                                        Accept: "application/json"
                                    };
                                    r.Authorization = "Bearer " + c.token, e.data || (e.data = {});
                                    var t = JSON.stringify(e.data),
                                        a = s.sign(t);
                                    r["X-Timestamp"] = a.timestamp, r["X-Sign"] = a.sign, r["X-Version-Code"] = n.versionCode;
                                }
                            });
                    }
                }, u);
            });
        };
    exports.addInterceptor = u;
}, {
    isPage: false,
    isComponent: false,
    currentFile: 'common/interceptor.js'
});
```

直接定位目标api

```

},
I = function () {
  if (y.mobile) {
    if (!y.timer) {
      var e;
      e = {
        mobile: y.mobile,
        marker: "user.passport.login"
      }, p.request("/miniprogram/sms.captcha/submit", e).then((function (e) {
        if (null == e ? void 0 : e.err) return i.index.showToast({
          title: null == e ? void 0 : e.msg,
          icon: "error"
        });
        y.timer = setInterval((function () {
          y.count--, y.codetext = y.count + " 秒后重新获取", y.count <= 0 && (clearInter
        )), 1e3)
      })))
    }
  } else i.index.showToast({
    title: "请输入手机号",
    icon: "none",
    mask: !0
  })
})

```

可以知道，request执行传递的参数为e和api，也就是{mobile: y.mobile,marker: "user.passport.login"}

分析代码，request执行逻辑，看到p=require(xx./request.js)

```

var e = require("../..@babel/runtime/helpers/defineProperty"),
    n = require("../..@babel/runtime/helpers/regeneratorRuntime"),
    o = require("../..@babel/runtime/helpers/asyncToGenerator"),
    t = require("../..@babel/runtime/helpers/objectSpread2"),
    i = require("../..common/vendor.js"),
    r = require("../..common/assets.js"),
    a = require("../..common/config.js"),
    c = require("../..stores/useAuth.js"),
    s = require("../..stores/useMate.js"),
    u = require("../..stores/useConf.js"),
    d = require("../..api/user/passport.js"),
    p = require("../..common/request.js"),
    l = require("../..stores/useClient.js");

exports.request = function (s) {
    var u = o.getStorage("freego", !1) || {},
        l = function (o, a) {
            s.success = function (e) {
                console.log("request success callback:%o", e), c(e, s) ? e.data && e.data.result && -3
                    t = !1, s.data = i(s.data), wx.request(s), n.forEach((function (e, o) {
                        e.data = i(e.data), wx.request(e)
                    })))
            }).catch((function () {
                t = !1
            }))) : o({
                data: {
                    retcode: -102,
                    retmsg: "login abort"
                }
            }) : o(e) : a(e)
        }, s.fail = function (e) {
            a(e)
        }, s.url = /^http/g.test(s.url) ? s.url : e.request.api_prefix + s.url, s.data = i(s.data),
        cookie: u.cookie || "",
        "Content-Type": "application/json"
    }, console.log("request options:%o", s), wx.request(s)
    };
    return new Promise((function (e, o) {
        a(s).then((function () {
            l(e, o)
        })).catch((function () {
            d().then((function () {
                l(e, o)
            })).catch((function (e) {
                console.log("login error %o", e), wx.showModal({
                    content: "登录失败",
                    showCancel: !1
                })
            })
        })
    )
    )
    )
    )

```

可以看到，export.request=function(s),其中的s就是url+data的合体，然后wx.request执行的时候传递的参数就是s，再去看interceptor

죄

現

--	--

存在

```

i = function (e) {
  var o = u();
  return Object.assign(e || {}, {
    vid: o.vid,
    skey: o.skey
  });
},
c = function (e, o) {
  var t;
  return 200 === e.statusCode || (t = /^5\d+/.test(e.statusCode) ? "系统繁忙, 请稍后重试" : "请
    showCancel: !1,
    title: "网络异常",
    content: t,
    confirmText: "确定"
  )), !1)
},
a = function (e) {
  return new Promise((function (o, t) {
    u().skey || !0 === e.ignore ? o() : t(new Error("empty session"))
  }));
};
exports.checkLogin = a;
var u = function () {
  return o.getStorage("session", !1) || {}
},

```

‘方法就是将e也就是s.data进行填充vid和skey重新组成json进行返回

vid和skey的获取是通过wx.getStorage机制读取缓存中的session，提取其中的vid和skey进行整合

对于首次而言肯定是不存在缓存的，所以会形成一种json作为s.data从而二进行加密
 {mobile: y.mobile, marker: "user.passport.login", vid: undefined, skey: undefined}

好的，加下来介绍下加密算法

```

common/sign.js", function (require, module, exports, window, document, frames, self, location, navigator, loca
strict";
t = require("./vendor.js");
rts.sign = function (n) {
var r = Date.now().toString().slice(0, 11);
return {
  sign: t.Md5(t.sha256("".concat(n, "&").concat(r))).toString(t.CryptoJS.enc.Hex),
  timestamp: r
}
}

```

获取当前时间戳的前11为作为timestamp进行返回

对加密数据拼接&和时间戳进行sha256加密，然后对加密结果进行md5加密，随后返回16进制作作为sign

根据这个逻辑，删除小程序缓存，进行抓取验证码数据包

本地写该算法，计算结果，对比数据包中结果

传递进来的参数s也就是request的options，可以看到是 {mobile:xxx,}